LeetCode Practice

5 Easy: Matrix

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

5
EASY

Patterns (in order)

1. Matrix (5)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

5 questions | tap a title to jump to it

Easy questions (5) Go >

Matrix (5) Go >

E	1. Valid Word Square	LC 422
E	2. Reshape the Matrix	LC 566
E	3. Toeplitz Matrix	LC 766
E	4. Transpose Matrix	LC 867
E	5. Largest Local Values in a Matrix	LC 2373

Easy questions (5)

EASY | TOPIC 1 OF 1

Matrix

5 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Valid Word Square LC 422
2. Reshape the Matrix LC 566
3. Toeplitz Matrix LC 766
4. Transpose Matrix LC 867
5. Largest Local Values in a Matrix LC 2373

EASY | Matrix | LeetCode #422 | 1 of 5 | 1/5

Valid Word Square

QUESTION

Given an array of strings words, return true if it forms a valid word square.

A sequence of strings forms a valid word square if the kth row and column read the same string, where 0 <= k < max(numRows, numColumns).

Example 1

Input	`words = ["abcd","bnrt","crmy","dtye"]`
Output	`true`
Explanation	*The 1st row and 1st column both read "abcd". The 2nd row and 2nd column both read "bnrt". The 3rd row and 3rd column both read "crmy". The 4th row and 4th column both read "dtye". Therefore, it is a valid word square.*

a	b	c	d
b	n	r	t
c	r	m	y
d	t	y	e

Example 2

Input	`words = ["abcd","bnrt","crm","dt"]`
Output	`true`
Explanation	*The 1st row and 1st column both read "abcd". The 2nd row and 2nd column both read "bnrt". The 3rd row and 3rd column both read "crm". The 4th row and 4th column both read "dt". Therefore, it is a valid word square.*

QUESTION

a	b	c	d
b	n	r	t
c	r	m	
d	t		

Example 3

Input	`words = ["ball","area","read","lady"]`
Output	`false`
Explanation	*The 3rd row reads "read" while the 3rd column reads "lead". Therefore, it is NOT a valid word square.*

b	a	l	l
a	r	e	a
r	e	a	d
l	a	d	y

CONSTRAINTS

- 1 <= words.length <= 500
- 1 <= words[i].length <= 500
- words[i] consists of only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

A word square means the kth row and kth column read the same string.

Equivalently: words[i][j] == words[j][i] for every valid (i,j). Right?

Words can have different lengths — shorter words mean some cells don't exist,

and that's fine as long as the column doesn't expect a character the row doesn't have."

"['abcd','bnrt','crmy','dtye']: row0='abcd', col0='abcd' ok etc. -> true ok

['ball','area','read','lady']: row2='read', col2='lead' -> false ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
build the transpose of the word list as strings
return it equals the original list
```

TIME	SPACE
O(total characters)	O(total characters)

NOTE

Not slow in time; the transpose copies every character. Comparing words[i][j] with words[j][i] directly (checking the bounds) needs O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Check row lengths + first column all same length — O(mn). Walk-through:"

`r = number of rows. c = number of columns = len(words[0]).`

For each word: if len(word) != c: return False. <- all rows same length

For col in 0..c-1:

`col_len = number of rows that have a character at col.`

Actually: all words have same length (checked). So column length = r.

Check: all words[row][col] form a valid word... wait.

Actually: words form a word square if words[i][j] == words[j][i] for all i,j.

Check: for each i,j: words[i][j] == words[j][i].

WHY transpose: a word square reads the same across and down -> it equals its transpose.

Time: O(mn). Space: O(1).

PHASE 4 CLEAN CODE

```python
class _422_ValidWordSquare:
    def validWordSquare(self, words: List[str]) -> bool:
        for row, word in enumerate(words):
            for col, char in enumerate(word):
                if col >= len(words) or row >= len(words[c
ol]) or words[col][row] != char:
                    return False
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the grid/table logic and show how each cell depends on previous cells."

words=['abcd','bnrt','crmy','dtye']

row=0,word='abcd': col=0->words[0][0]='a'=='a'ok. col=1->words[1][0]='b'=='b'ok. etc.

All rows pass symmetrically -> True ok

PHASE 5 TEST & DEBUG

words=['ball','area','read','lady']

row=2,word='read': col=0->words[0][2]='l'=='r'? No -> False ok

words=['abcd','bnrt','crm','dt'] (shorter words)

row=0,col=3: words[3][0]='d'=='d'ok. row=1,col=2: words[2][1]='r'=='n'? No wait —

words[2]='crm', words[2][1]='r'. words[1][2]='r' ok. All pass -> True ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*m)	O(1)

The two guard conditions are the subtle part:

col >= len(words) means the column index is out of bounds — the row has more chars

than there are rows, which breaks symmetry -> False.

row >= len(words[col]) means the transpose cell doesn't exist -> False.

Given more time I'd ask: what if we want to BUILD a word square from a word list?

That's a backtracking problem — place words row by row, pruning based on column prefixes.

EASY | Matrix | LeetCode #566 | 2 of 5 | 2/5

Reshape the Matrix

QUESTION

In MATLAB, there is a handy function called reshape which can reshape an m x n matrix into a new one with a different size r x c keeping its original data.

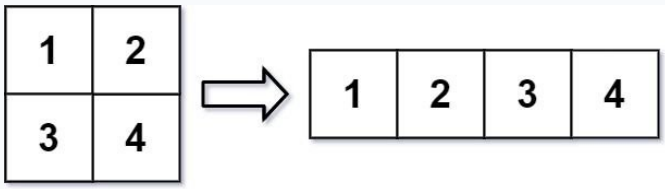
You are given an m x n matrix mat and two integers r and c representing the number of rows and the number of columns of the wanted reshaped matrix.

The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.

If the reshape operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

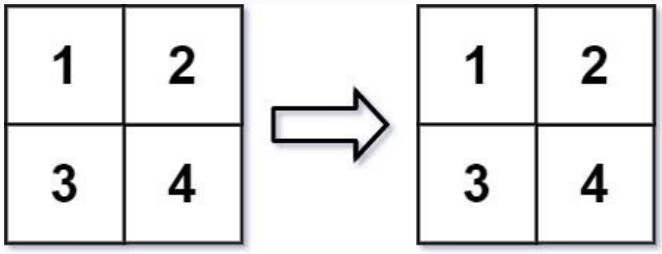
Example 1

Input	`mat = [[1,2],[3,4]], r = 1, c = 4`
Output	`[[1,2,3,4]]`



Example 2

Input	`mat = [[1,2],[3,4]], r = 2, c = 4`
Output	`[[1,2],[3,4]]`



CONSTRAINTS

- m == mat.length
- n == mat[i].length
- 1 <= m, n <= 100
- 1000 <= mat[i][j] <= 1000
- 1 <= r, c <= 300

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We flatten the original m×n matrix in row-major order and refill into an r×c matrix.

If m*n != r*c the reshape is impossible — return the original matrix unchanged. Right?"

"Quick questions:

Values can be negative — fine, we're just moving them.

r and c can be up to 300 even though m,n are up to 100 — just need total elements to match."

"[[1,2],[3,4]], r=1,c=4: 4 elements total, 1*4=4 -> [[1,2,3,4]] ok

[[1,2],[3,4]], r=2,c=4: 4 elements total, 2*4=8 != 4 -> return original ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
flatten the matrix into a list
fill the new r x c matrix from the list in order
```

TIME	SPACE
O(m * n)	O(m * n)

NOTE

Not slow: every element is moved once. The flat list is an extra copy; reading old[k / n][k % n] for k = 0 .. m*n - 1 writes straight into the new matrix.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Index math — O(mn). Walk-through:"

First validate: if m*c != r*n (total elements differ) -> return [].

For flat index k in 0..m*n-1:

Source cell: (k // n, k % n).

Destination cell: (k // c, k % c).

ans[k//c][k%c] = mat[k//n][k%n].

WHY flat index: reshape is just reinterpreting the same flat sequence in a different 2-D shape.

Reading row-major from source and writing row-major to destination = correct reshape.

No intermediate array needed if we map source index to dest index directly.

Time: O(mn). Space: O(1) extra (output matrix is required).

PHASE 4 CLEAN CODE

```python
class _566_ReshapeMatrix:
    def matrixReshape(self, mat: List[List[int]], r: int,
c: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        result = [[0] * c for _ in range(r)]
        for k in range(m * n):
            result[k // c][k % c] = mat[k // n][k % n]
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace matrixReshape([[1,2],[3,4]], r=1, c=4)."

Flatten: [1,2,3,4]. Total=4. r*c=4=total -> valid reshape

Fill new matrix row by row: result[0]=[1,2,3,4] ok

Index mapping: original[i//c_orig][i%c_orig] -> result[i//c_new][i%c_new]

i=0->(0,0)=1, i=1->(0,1)=2, i=2->(1,0)=3, i=3->(1,1)=4

result=[[1,2,3,4]] ok

Edge: r*c != m*n -> return original matrix (impossible reshape).

WHY single index: eliminates nested loop logic.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(r*c)** for the output matrix — unavoidable

The index formula k//n and k%n is the core insight: flatten index -> 2D source.

Given more time I'd ask: what if the matrix is too large to fit in memory?

Process row by row, streaming elements into the new shape.

EASY | Matrix | LeetCode #766 | 3 of 5 | 3/5

Toeplitz Matrix

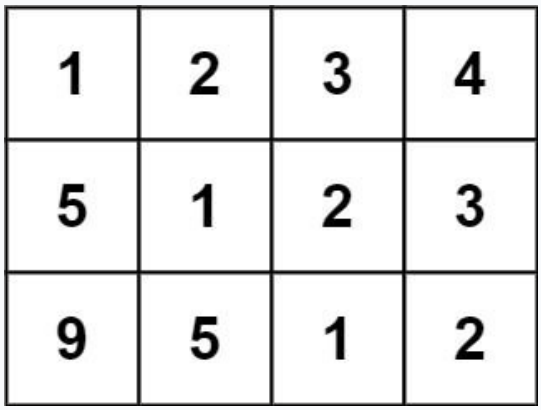
QUESTION

Given an m x n matrix, return true if the matrix is Toeplitz. Otherwise, return false.

A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements.

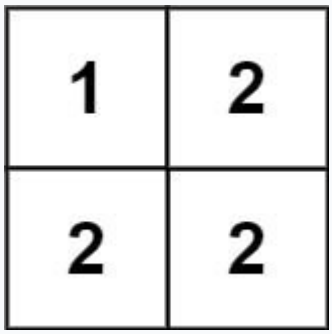
Example 1

Input	`matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]`
Output	`true`
Explanation	*In the above grid, the diagonals are: "[9]", "[5, 5]", "[1, 1, 1]", "[2, 2, 2]", "[3, 3]", "[4]". In each diagonal all elements are the same, so the answer is True.*



Example 2

Input	`matrix = [[1,2],[2,2]]`
Output	`false`
Explanation	*The diagonal "[1, 2]" has different elements.*



CONSTRAINTS

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 20
- 0 <= matrix[i][j] <= 99
- Follow up:
  - What if the matrix is stored on disk, and the memory is limited such that you can only load at most one row of the matrix into the memory at once?
  - What if the matrix is so large that you can only load up a partial row into the memory at once?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

A Toeplitz matrix has the same value along every top-left to bottom-right diagonal.

We need to check every cell (except the last row and column) equals its down-right neighbour.

Equivalently: matrix[i][j] == matrix[i+1][j+1] for all valid i,j. Right?"

"[[1,2,3,4],[5,1,2,3],[9,5,1,2]]: each cell equals its down-right neighbour -> true ok

[[1,2],[2,2]]: matrix[0][0]=1 != matrix[1][1]=2 -> false ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (a, b) in pairs(cells):
    check that they are equal
```

TIME	SPACE
O(m * n * min(m, n))	O(1)

WHY IT IS SLOW

Compares all pairs on each diagonal. Every cell must equal its up-left neighbour: one pass, O(m * n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Check each cell against its upper-left neighbor — O(mn). Walk-through:"

A Toeplitz matrix: every diagonal has the same value.

Check: for each (i,j) where i>0 and j>0: matrix[i][j] == matrix[i-1][j-1].

WHY only one neighbor: if every element equals its upper-left neighbor, then

the entire diagonal is consistent (transitivity: a[1][1]==a[0][0], a[2][2]==a[1][1] -> a[2][2]==a[0][0]).

Process: first row and first column are always valid (no upper-left to compare to).

Return False as soon as any mismatch found.

Time: O(mn). Space: O(1).

PHASE 4 CLEAN CODE

```
class _766_ToeplitzMatrix:
    def isToeplitzMatrix(self, matrix: List[List[int]]) ->
 bool:
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows - 1):
            for j in range(cols - 1):
                if matrix[i][j] != matrix[i + 1][j + 1]:
                    return False
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace isToeplitzMatrix on [[1,2,3,4],[5,1,2,3],[9,5,1,2]]."

Each diagonal top-left to bottom-right must have equal elements

Check: matrix[i][j]==matrix[i-1][j-1] for all i>0, j>0

(1,1)=1 vs (0,0)=1 ok. (1,2)=2 vs (0,1)=2 ok. (1,3)=3 vs (0,2)=3 ok

(2,1)=5 vs (1,0)=5 ok. (2,2)=1 vs (1,1)=1 ok. (2,3)=2 vs (1,2)=2 ok

All pass -> return True ok

WHY [i][j]==[i-1][j-1]: same diagonal means row-col difference is constant; adjacent elements differ by (1,1).

Edge: 1x1 matrix -> always True; 1xN or Nx1 -> always True.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	O(1)

Follow-up 1: if we can only load one row at a time, compare consecutive rows

by checking row[j] == prev_row[j+1] for all j — same logic, streaming.

Follow-up 2: if we can only load partial rows, compare overlapping chunks of

adjacent rows. Both follow-ups keep O(1) extra space.

Given more time I'd ask: can we verify a diagonal in O(1) per cell?

Yes — the current approach already does exactly that.

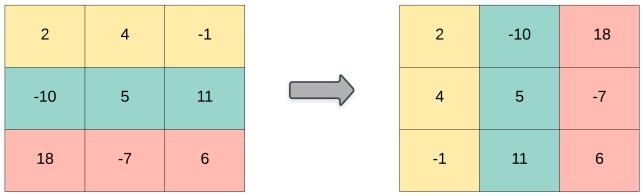
EASY | Matrix | LeetCode #867 | 4 of 5 | 4/5

Transpose Matrix

QUESTION

Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.



Example 1	
Input	`matrix = [[1,2,3],[4,5,6],[7,8,9]]`
Output	`[[1,4,7],[2,5,8],[3,6,9]]`

Example 2	
Input	`matrix = [[1,2,3],[4,5,6]]`
Output	`[[1,4],[2,5],[3,6]]`

CONSTRAINTS

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 1000
- 1 <= m * n <= 10^5
- 10^9 <= matrix[i][j] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

The transpose swaps rows and columns: element at (r,c) moves to (c,r).

If the original is m×n, the result is n×m. Return a new matrix — not in-place. Right?"

"[[1,2,3],[4,5,6],[7,8,9]] -> [[1,4,7],[2,5,8],[3,6,9]] ok

[[1,2,3],[4,5,6]] (2×3) -> [[1,4],[2,5],[3,6]] (3×2) ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
make a new n x m matrix
for cell in cells:
    new[j][i] = old[i][j]
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(m * n)	O(m * n)

NOTE

Not slow: every element is moved once, which is optimal. For a square matrix it can be done in place by swapping across the diagonal.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Index swap — O(mn). Walk-through:"

Result has dimensions n×m (cols become rows).

`result[j][i] = matrix[i][j] for all i,j.`

Python shortcut: list(zip(*matrix)) transposes to list of tuples.

Convert each tuple to list for mutability.

WHY col count = original row count: transpose flips dimensions.

Result[j] = original column j (now a row).

Time: O(mn). Space: O(mn) for output."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

```
class _867_TransposeMatrix:
    def transpose(self, matrix: List[List[int]]) -> List[L
ist[int]]:
        rows, cols = len(matrix), len(matrix[0])
        result = [[0] * rows for _ in range(cols)]
        for r in range(rows):
            for c in range(cols):
                result[c][r] = matrix[r][c]
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace transpose on [[1,2,3],[4,5,6],[7,8,9]]."

`result[j][i] = matrix[i][j]`

`result[0] = [matrix[0][0],matrix[1][0],matrix[2][0]] = [1,4,7]`

`result[1] = [matrix[0][1],matrix[1][1],matrix[2][1]] = [2,5,8]`

`result[2] = [matrix[0][2],matrix[1][2],matrix[2][2]] = [3,6,9]`

Return [[1,4,7],[2,5,8],[3,6,9]] ok

Trace non-square [[1,2,3],[4,5,6]]: rows=2,cols=3 -> result is 3×2

result[0]=[1,4], result[1]=[2,5], result[2]=[3,6] ok

WHY zip(*matrix): Python idiom that transposes; equivalent to nested loop.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** for the output

Python one-liner: return [list(row) for row in zip(*matrix)]

```
zip(*matrix) unpacks rows as arguments to zip, which
  pairs corresponding columns.
```

Elegant and interview-clean — worth mentioning as an alternative.

Given more time I'd ask: can we transpose a square matrix in-place?

Yes — swap matrix[i][j] and matrix[j][i] for all i < j. O(1) extra space.

IN YOUR APP | Expose | Editorial | Community

EASY | Matrix | LeetCode #2373 | 5 of 5 | 5/5

Largest Local Values in a Matrix

QUESTION

You are given an n x n integer matrix grid.

Generate an integer matrix maxLocal of size (n - 2) x (n - 2) such that:

CONSTRAINTS

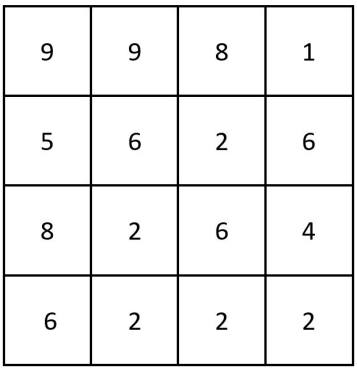
- maxLocal[i][j] is equal to the largest value of the 3 x 3 matrix in grid centered around row i + 1 and column j + 1.

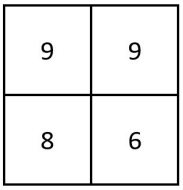
In other words, we want to find the largest value in every contiguous 3 x 3 matrix in grid.

Return the generated matrix.

Example 1

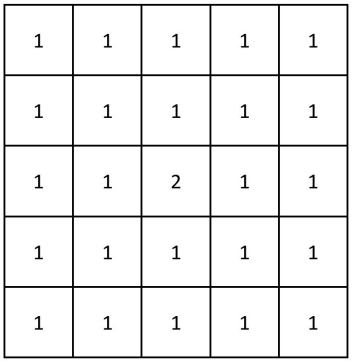
Input	`grid = [[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]`
Output	`[[9,9],[8,6]]`
Explanation	*The diagram above shows the original matrix and the generated matrix. Notice that each value in the generated matrix corresponds to the largest value of a contiguous 3 x 3 matrix in grid.*

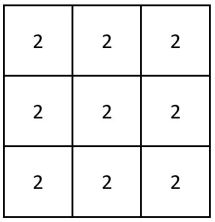




Example 2

Input	`grid = [[1,1,1,1,1],[1,1,1,1,1],[1,1,2,1,1],[1,1,1,1,1],[1,1,1,1,1]]`
Output	`[[2,2,2],[2,2,2],[2,2,2]]`
Explanation	*Notice that the 2 is contained within every contiguous 3 x 3 matrix in grid.*





QUESTION

- n == grid.length == grid[i].length
- 3 <= n <= 100
- 1 <= grid[i][j] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

For each cell (i,j) in the output (n-2)×(n-2) matrix,

find the max in the 3×3 subgrid of the input centered at (i+1, j+1). Right?

Input is always n×n square, n >= 3."

"grid=[[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]] (4×4):

output is 2×2. Top-left 3×3 center=(1,1): max of [[9,9,8],[5,6,2],[8,2,6]]=9.

Top-right 3×3 center=(1,2): max of [[9,8,1],[6,2,6],[2,6,4]]=9. etc. -> [[9,9],[8,6]] ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    # of (n - 2) x (n - 2) result
    take the maximum of the 3 x 3 block
```

TIME	SPACE
O(n^2)	O(n^2)

NOTE

Not slow: nine comparisons per cell, so O(9 * n^2) = O(n^2), and the output has to be that size. The only trick is the indexing.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding 3x3 window max check — O(mn). Walk-through:"

For each interior cell (i,j) where 1 <= i <= m-2 and 1 <= j <= n-2:

Find max of the 3x3 grid centered at (i,j).

result[i-1][j-1] = max of grid[i-1..i+1][j-1..j+1]. <- offset by 1 since border excluded

Return result (size (m-2)×(n-2)).

WHY interior only: 3x3 window centered at border would go out of bounds.

WHY 9-cell max: 'largest local value' = max of the 3x3 neighborhood.

Brute: O(mn * 9) = O(mn). Can optimize with 2-D sliding window max but O(mn) already fine.

Time: O(mn). Space: O((m-2)*(n-2)) for output.

PHASE 4 CLEAN CODE

```
class _2373_LargestLocal:
    def largestLocal(self, grid: List[List[int]]) -> List[
List[int]]:
        n      = len(grid)
        result = [[0] * (n - 2) for _ in range(n - 2)]
        for i in range(n - 2):
            for j in range(n - 2):
                for di in range(3):
                    for dj in range(3):
                        result[i][j] = max(result[i][j], g
rid[i + di][j + dj])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace largestLocal([[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]])."

For each center cell (i,j) where 1<=i,j<=n-2: find max in 3x3 neighborhood

n=4. Centers: (1,1),(1,2),(2,1),(2,2)

(1,1): neighborhood rows 0-2, cols 0-2: [9,9,8,5,6,2,8,2,6] -> max=9

(1,2): rows 0-2, cols 1-3: [9,8,1,6,2,6,2,6,4] -> max=9

(2,1): rows 1-3, cols 0-2: [5,6,2,8,2,6,6,2,2] -> max=8

(2,2): rows 1-3, cols 1-3: [6,2,6,2,6,4,2,2,2] -> max=6

Return [[9,9],[8,6]] ok

WHY brute force 3x3: n<=100, total work O(9*n²) = O(n²) — fast enough.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(n²)**
outer loops O((n-2)²) ~ O(n²), inner loops constant 9 operations	for the output

Given more time I'd ask: what if the window size is variable (k×k instead of 3×3)?

For large k, recomputing the max from scratch is O(k²) per cell -> O(n²k²) total.

A 2D sliding window maximum using monotonic deques brings it down to O(n²).